PyCon 2015

Montréal • April 8-16

Facts and Myths about Python names and values

Ned Batchelder

Google Microsoft MANDRILL Dropbox rackspace the open cloud company

Facts and Myths about Names and Values

Ned Batchelder
@nedbat

http://bit.ly/pynames1



OPENedX

Educating the world

All open source

http://open.edx.org

Open space: 3:45 today 510B

bit.ly / pynames1



Python is simple

- Works like other languages
- ... until it doesn't
- Mechanisms are simple
- ... but the effects can be surprising
- Names, values, assignment, mutability

bit.ly / pynames1



Names refer to values

Assignment: make this name refer to that value

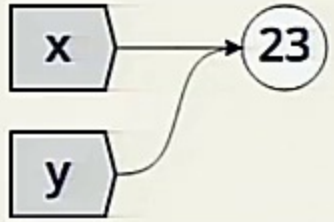
```
x = 23
print(x)          # 23
```

bit.ly / pynames1



Many names can refer to one value

```
x = 23
y = x
```

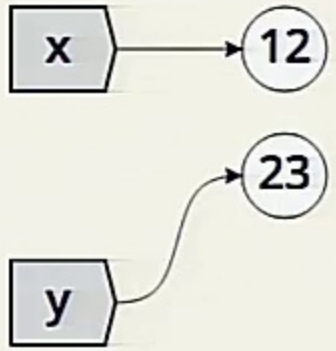


bit.ly / pynames1



Names are reassigned independently

```
x = 23
y = x
x = 12
```

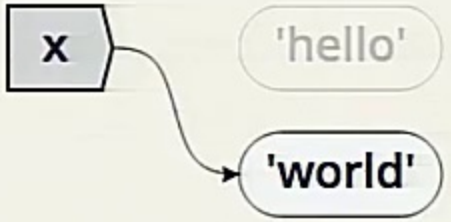


bit.ly / pynames1



Values live until no references

```
x = "hello"
x = "world"
```



bit.ly / pynames1



Assignment never copies data

```
nums = [1, 2, 3]
other = nums
```

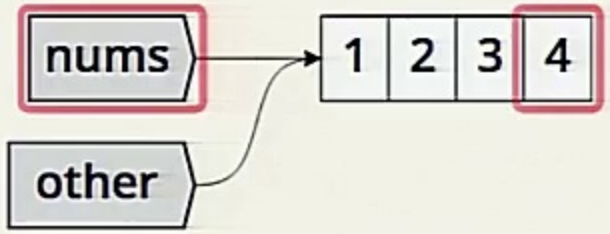


bit.ly / pynames1



Changes are visible through all names

```
nums = [1, 2, 3]
other = nums
nums.append(4)
print(other)          # [1, 2, 3, 4] !!!
```



bit.ly / pynames1



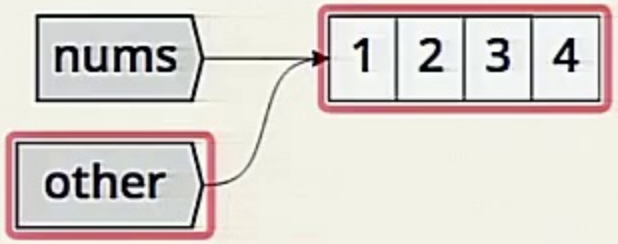
Mutable aliasing!

- A mutable value



Changes are visible through all names

```
nums = [1, 2, 3]
other = nums
nums.append(4)
print(other)          # [1, 2, 3, 4] !!!
```



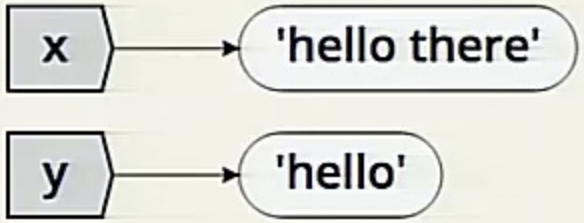
bit.ly / pynames1



Immutable values can't alias

Immutable types: ints, floats, strings, tuples

```
x = "hello"
y = x
x = x + " there"
```



bit.ly / pynames1



"Change" is unclear

bit.ly / pynames1



"Change" is unclear

Changing an int: **rebinding**

```
x = x + 1
```

Changing a list: **mutating**

```
nums.append(7)
```

Can also **rebind** lists:

```
nums = nums + [7]
```

Can't mutate an int: ints are **immutable**



Mutable and immutable are assigned the same

- Assignment is the same for **all** values.

bit.ly / pynames1



Assignment variants

```
x += y
x = x + y              # conceptually
x = x.__iadd__(y)      # actually!
```

bit.ly / pynames1



Assignment variants

```
x += y
x = x + y              # conceptually
x = x.__iadd__(y)      # actually!
```

```
# pseudo-code!
class List:
    def __iadd__(self, other):
        self.extend(other)
        return self
```

```
# These are the same:
num_list += [4, 5]
num_list.extend([4, 5]);  num_list = num_list
```



References can be more than just names

List elements are references

```
nums = [1, 2, 3]
x = nums[1]
```



bit.ly / pynames1



Lots of things are references

- Object attributes
- List elements
- Dict values (and keys!)
- Anything on the left side of an assignment

```
my_obj.attr = 23
my_dict[key] = 24
my_list[index] = 25
my_obj.attr[key][index].attr = "etc, etc"
```



Lots of things are assignments

All of these assign to **X**

```
X = ...
for X in ...
class X(...):
def X(...):
def fn(X):
import X
from ... import X
except ... as X:
with ... as X:
```



bit.ly / pynames1

For loops

```
for x in sequence:
    something(x)
```

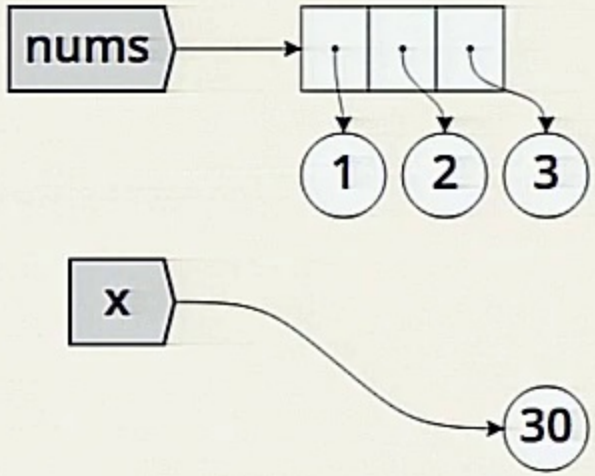
```
x = sequence[0]
something(x)
x = sequence[1]
something(x)
...
```



For loops

⇒ Loop Like a Native (PyCon 2013): bit.ly/pyiter

```
nums = [1, 2, 3]
for x in nums:              # x = nums[0] ...
    x = x * 10
print(nums)                 # [1, 2, 3]   :(
```





Function arguments are assignments

```
def func(x):      # x = num
    print(x)
    return

num = 17
func(num)         # x = num
print(num)
```

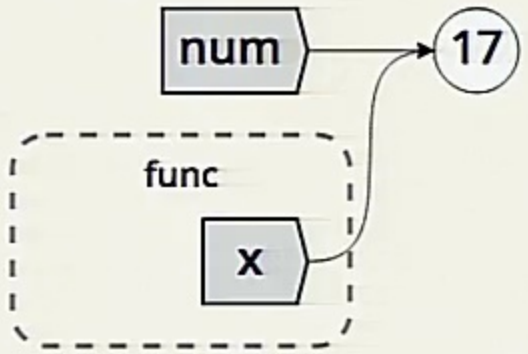
bit.ly / pynames1



Function arguments are assignments

```
def func(x):      # x = num
    print(x)
    return

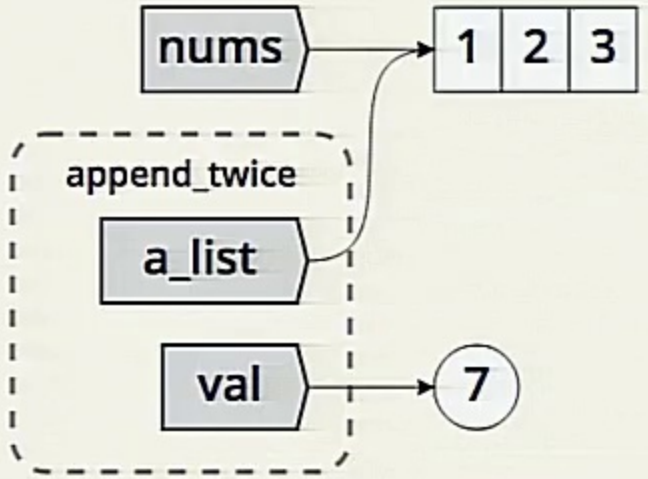
num = 17
func(num)         # x = num
print(num)
```





```
def append_twice(a_list, val):
    a_list.append(val)
    a_list.append(val)
    return

nums = [1, 2, 3]
append_twice(nums, 7)
print(nums)          # [1, 2, 3, 7, 7]
```

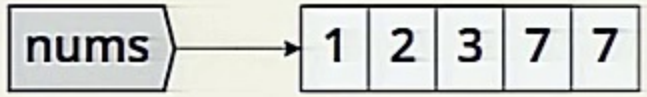


bit.ly / pynames1



```
def append_twice(a_list, val):
    a_list.append(val)
    a_list.append(val)
    return

nums = [1, 2, 3]
append_twice(nums, 7)
print(nums)           # [1, 2, 3, 7, 7]
```

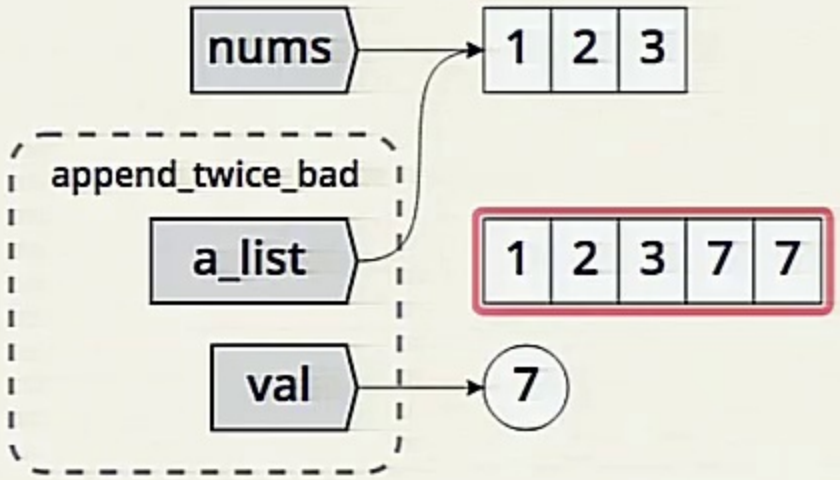


bit.ly / pynames1



```
def append_twice_bad(a_list, val):
    a_list = a_list + [val, val]
    return

nums = [1, 2, 3]
append_twice_bad(nums, 7)
print(nums)          # [1, 2, 3]
```



bit.ly / pynames1



```
def append_twice_bad(a_list, val):
    a_list = a_list + [val, val]
    return

nums = [1, 2, 3]
append_twice_bad(nums, 7)
print(nums)              # [1, 2, 3]
```

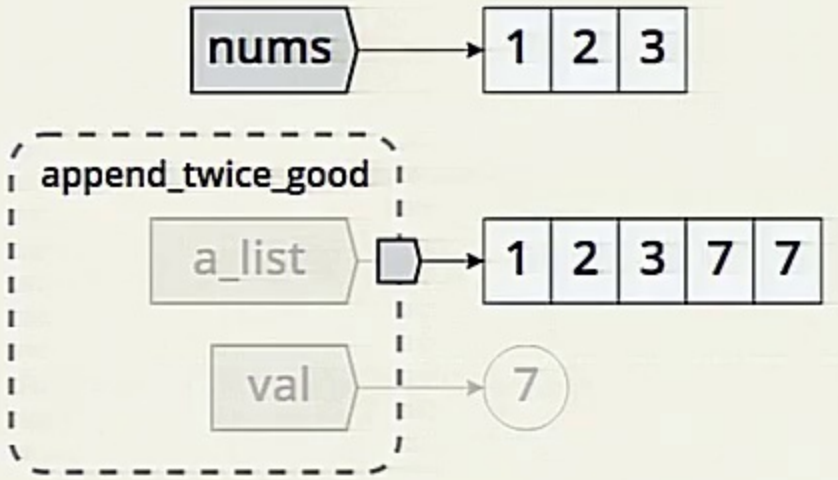


bit.ly / pynames1



```
def append_twice_good(a_list, val):
    a_list = a_list + [val, val]
    return a_list

nums = [1, 2, 3]
nums = append_twice_good(nums, 7)
print(nums)          # [1, 2, 3, 7, 7]
```



bit.ly / pynames1



Three append_twice functions

```
def append_twice(a_list, val):
    """Mutates argument."""
    a_list.append(val)
    a_list.append(val)
```

```
def append_twice_bad(a_list, val):
    """Useless!"""
    a_list = a_list + [val, val]
```

```
def append_twice_good(a_list, val):
    """Returns new list."""
    a_list = a_list + [val, val]
    return a_list
```

bit.ly / pynames1



Any name → any value @ any time

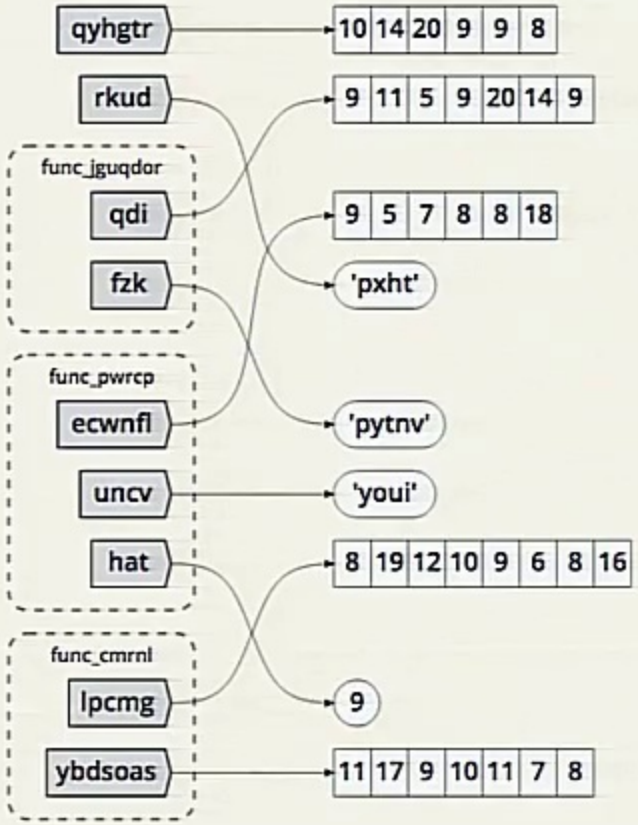
Dynamic typing

```
x = 12
x = "hello"
x = [1, 2, 3]
x[1] = "two"
```



Names have no type

Values have no scope



bit.ly / pynames1



Other topics

- "Python has no variables." **Ugh!**



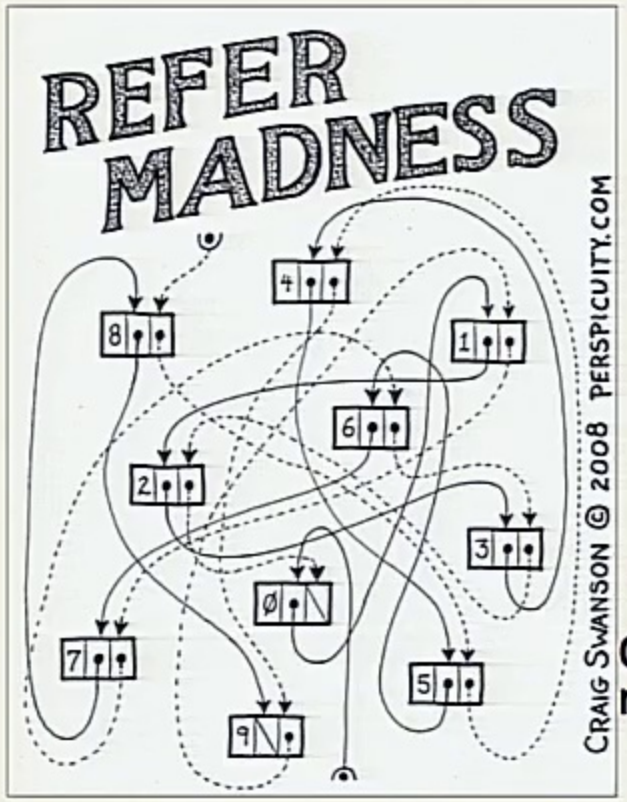
Other topics

- "Python has no variables." **Ugh!**
- Call-by-value or call-by-reference?
- Making a 2D list:

```
board = [[0] * 8] * 8                    # bad
board = [[0] * 8 for _ in range(8)]      # good
```




Questions?



Thanks, Craig Swanson http://perspicuity.com

@ nedbat

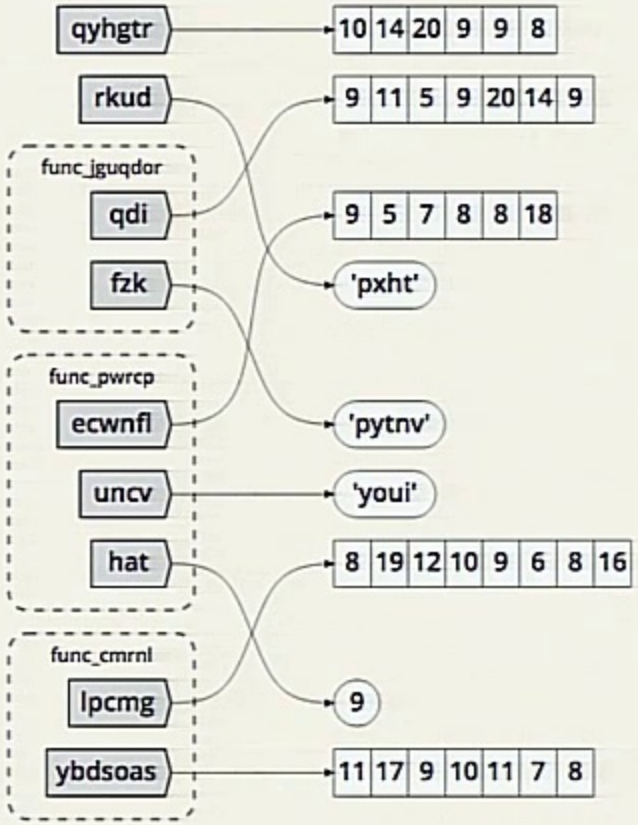
http:// bit.ly / pynames1

Open edX open space: 3:4
room 510B



Names have no type

Values have no scope



bit.ly / pynames1

